Lecture 26 — NP-Completeness (Part 2)

COMPSCI 308 — Design and Analysis of Algorithms

Xing Shi Cai https://newptcai.gitlab.io

Duke Kunshan University, Kunshan, China

Jan-Mar 2026

Summary

ITA 34.1 P — Polynomial Time (Continued)

A Formal-Language Framework

ITA 34.2 Polynomial-Time Verification

Verification Algorithms

The Complexity Class NP

ITA 34.3 NP-Completeness and Reducibility

Reducibility

Assignments[1]

Practice makes perfect!

📖 Required Readings:

- Section 34.2

✏️ Required Exercises:

- Exercises 34.2: 1–4.
- Exercises in this slide deck.

[1] 💣 Assignments will not be collected; however, quiz problems will be selected from them. (This includes both Readings and Exercises.)

ITA 34.1 P — Polynomial Time (Continued)

ITA 34.1 P — Polynomial Time (Continued)

A Formal-Language Framework

Formal Language Theory

Formal language theory gives us a precise way to define and reason about decision problems.

Some definitions:

- An alphabet Σ is a finite set of symbols.

The alphabet of English is $\Sigma = \{a, b, \ldots, z, A, B, \ldots, Z\}$.

What is Σ for Chinese?

{张, 蔡, 丁, 陈, 成, 朱, 正, ...}

Formal Language Theory

Some definitions:

- Σ^* denotes the set of all finite strings of symbols from Σ, including the empty string ϵ.
- A language L over Σ is any subset of Σ^*.

📌 If $\Sigma = \{$🍎$\}$, then

$\Sigma^* = \{\epsilon,$ 🍎, 🍎🍎, 🍎🍎🍎, ... $\}$

empty

🍰 What is Σ^* if $\Sigma = \{0,1\}$?

All binary strings.

Some definitions:

- A language L over Σ is any subset of Σ^*.

📌 With $\Sigma = \{$🍎$\}$, one possible language is

$$L = \{🍎, 🍎🍎🍎\}$$

📌 With $\Sigma = \{a, b, \ldots, z\}$, one possible language is

$$L = \{\text{amazing, wonderful, professor, cai}\}$$

🍰 Can you give a language with $\Sigma = \{0, 1\}$?

$$L = \{0, 01, 11, 110, 101\}$$

$L = \{0, 00, 000, \ldots\} = \{\text{strings of 0's}\}$

Some definitions:

- The complement of L is $\overline{L} = \Sigma^* - L$.

📌 If $\Sigma = \{0,1\}$ and

empty string

$$L = \{1\}^* = \{\epsilon, 1, 11, 111, \dots\} = \{\text{strings of 1's and } \varepsilon\}$$

then

$$\overline{L} = \{w \in \{0,1\}^* : w \text{ contains at least one } 0\}$$

Some definitions:

- The concatenation of two languages L_1 and L_2 is

$$L_1L_2 = \{x_1x_2 : x_1 \in L_1,\ x_2 \in L_2\}.$$

📌 If $L_1 = \{a\}$ and $L_2 = \{a, b, \ldots, z\}$, then

$$L_1L_2 = \{aa, ab, ac, \ldots, az\}$$ = { string contains two letters where the first letter is a }

🍰 If $L_1 = \{$蔡$\}$ and $L_2 = \Sigma \times \Sigma$ where Σ is the set of all Chinese characters, what is

$$L_1L_2 = \{\text{蔡醒侍, 蔡光辰} \ldots\}?$$

Chinese strings length 2.

Decision Problems and Languages

In formal language theory, the set of all instances of a decision problem Q is Σ^* where $\Sigma = \{0,1\}$, since every problem instance can be encoded as a binary string.

📌 For instance, if Q asks

Is the number i even?

then every natural number is an instance encoded as a binary string.

$Q = \{0, 10, 110, 100, \ldots\}$

$= \{$all binary strings ending with 0$\}$

So Q is a language over $\Sigma = \{0,1\}$.

Decision Problems and Languages

The instances of Q for which the answer is yes form a language

binary string

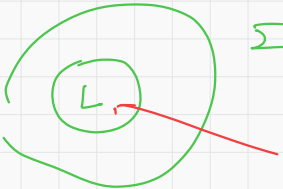
$$L = \{x \in \Sigma^* : Q(x) = 1\}.$$

$\varepsilon \notin L$.

For the problem "Is i even?", what is the language L?

$$L = \{x \in \Sigma^* : \text{The last bit of } x \text{ is } \underline{\quad 0 \quad}\}$$

$= \{0, 10, 110, 100, \ldots\}$.



Σ^*

Decision Problem which the answer is Yes.

Decision Problems and Languages

The instances of Q for which the answer is yes form a language

$$L = \{x \in \Sigma^* : Q(x) = 1\}.$$

standard encoding $\in \Sigma^*$

📌 This applies to more complex problems, such as:

$$\begin{aligned}\text{PATH} = \{\langle G, u, v, k\rangle : {} & G = (V, E) \text{ is an undirected graph,} \\ & u, v \in V, \quad k \geq 0 \text{ is an integer,} \\ & \text{and } G \text{ contains a path from } u \text{ to } v \text{ with at most } k \text{ edges}\}.\end{aligned}$$

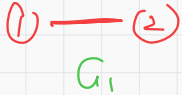
💡 An instance like $\langle G, u, v, k\rangle$ is always encodable as an element of Σ^*.

📌 Example: Decision Problem as Language

Consider the problem:

Is the graph G with two vertices connected?

The two possible adjacency matrices for G are:



$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \qquad \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

🍰 What do these two graphs look like?

$\langle G_1 \rangle = 0, 1, 1, 0$

$\langle G_2 \rangle = 0, 0, 0, 0.$

G_3

Adj matrix:

$$\begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

$\langle G_3 \rangle = 011\ 101\ 110.$

2-VERTEX-GRAPH $= \{0000, 0110\}$

$\langle G_3 \rangle \in$ 2-VERTEX-GRAPH.

Example: Decision Problem as Language

2-VERTEX-GRAPH

$= \{ 0000, 0110 \}$

$\langle G_2 \rangle$ $\langle G_1 \rangle$

Consider the problem:

Is the graph G with two vertices connected?

The two possible adjacency matrices for G are:

$\langle G_1 \rangle = 0,1,1,0$ — $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ — $\langle G_2 \rangle = 0,0,0,0.$

Encoding $\langle G \rangle$ as the binary string of its adjacency matrix, the problem becomes the language:

CONNECTED-2-VERTEX-GRAPH

$= \{\langle G \rangle : G$ is a connected undirected graph with 2 vertices$\}$

$= \{0110\}$. $\subseteq \Sigma^*$

Accepting and Rejecting a Language

The formal language framework connects decision problems to the algorithms that solve them.

An algorithm A accepts a string $x \in \{0,1\}^*$ if $A(x) = 1$.

A rejects x if $A(x) = 0$.

Accepting and Rejecting a Language

The formal language framework connects decision problems to the algorithms that solve them.

An algorithm A accepts a string $x \in \{0,1\}^*$ if $A(x) = 1$.

A rejects x if $A(x) = 0$.

The language accepted by A is

$$L = \{x \in \{0,1\}^* : A(x) = 1\}.$$

$\{x : A(x) = 1\}$

$=$ Even-numbers.

$= \{0, 10, 100, 110, \dots\}$.

🍰 If $A(x) = 1$ if and only if the last bit of x is 0, what language does A accept?

For example if A decides if x is a 2-vertex graph. The $L = \{x : A(x) = 1\} = \{0000, 0110\}$.

Decided vs. Accepted

Even if L is accepted by A—meaning A outputs 1 on every input in L—A may fail to reject inputs not in L. For example, A might loop forever on $x \notin L$.

$L = \{x : A(x) = 1\}$.

This does not imply that

$\overline{L} = \{x : A(x) = 0\}$

Decided vs. Accepted

Even if L is accepted by A—meaning A outputs 1 on every input in L—A may fail to *reject* inputs not in L. For example, A might loop forever on $x \notin L$.

A language L is decided by A if A handles *every* input correctly:

- A accepts every $x \in L$: $L = \{x \in \{0,1\}^* : A(x) = 1\}$
- A rejects every $x \notin L$: $\overline{L} = \{x \in \{0,1\}^* : A(x) = 0\}$

Decided vs. Accepted

Even if L is accepted by A—meaning A outputs 1 on every input in L—A may fail to *reject* inputs not in L. For example, A might loop forever on $x \notin L$.

A language L is decided by A if A handles *every* input correctly:

- A accepts every $x \in L$: $L = \{x \in \{0,1\}^* : A(x) = 1\}$ ← accept
- A rejects every $x \notin L$: $\overline{L} = \{x \in \{0,1\}^* : A(x) = 0\}$

} decide.

💡 If A decides L, then A also accepts L.

💣 The converse does not hold in general (e.g., Turing's Halting Problem has an accepting but no deciding algorithm).

Example: Decided vs. Accepted

Consider an algorithm that, given $\langle G, u, v, k \rangle$:

- runs BFS on G from u;
- outputs 1 if there is a path of at most k edges from u to v;
- otherwise, *never terminates*.

This only accepts. but not decide.

📌 Example: Decided vs. Accepted

Consider an algorithm that, given $\langle G, u, v, k \rangle$:

- runs BFS on G from u;
- outputs 1 if there is a path of at most k edges from u to v;
- otherwise, *never terminates*.

🍰 Does this algorithm accept the language

$$\text{PATH} = \{\langle G, u, v, k \rangle : G \text{ contains a path from } u \text{ to } v \text{ with at most } k \text{ edges}\}?$$

🍰 Does it decide PATH? No.

📌 Example: Decided vs. Accepted

Consider an algorithm that, given $\langle G, u, v, k \rangle$:

- runs BFS on G from u;
- outputs 1 if there is a path of at most k edges from u to v;
- otherwise, outputs 0.

🍰 Does this algorithm accept the language

$$\text{PATH} = \{\langle G, u, v, k \rangle : G \text{ contains a path from } u \text{ to } v \text{ with at most } k \text{ edges}\}?$$

🍰 Does it decide PATH? Yes.

Accepted and Decided in Polynomial Time

L is accepted in polynomial time by A if A accepts L and there exists a constant k such that for every string $x \in L$ of length n, A accepts x in $O(n^k)$ time.

Accepted and Decided in Polynomial Time

L is accepted in polynomial time by A if A accepts L and there exists a constant k such that for every string $x \in L$ of length n, A accepts x in $O(n^k)$ time.

L is decided in polynomial time by A if there exists a constant k such that for every string $x \in \{0,1\}^*$ of length n, A correctly decides whether $x \in L$ in $O(n^k)$ time.

If L is decided in polynomial time, then L is accepted in polynomial time.

Is the converse also true?

Decide $\Longrightarrow$ Accept.

L is accepted in poly-time $\rightarrow$ L is also decide in poly-time.

If A returns 1 in poly time. if $x \in L$.

can we find A' which ① returns 1 if $x \in L$.

② returns 0 if $x \notin L$.

$L = \{x : A(x) = 1\}$

Complexity Class P

Using formal language theory, we define

$$\mathsf{P} = \{L \subseteq \{0,1\}^* : \text{there exists an algorithm that decides } L \text{ in polynomial time}\}.$$

$= \{L :$ There exists A such that

$A(x) = 1$ if $x \in L$.

and $A(x) = 0$ if $x \notin L$.

in polynomial time?

$\subseteq \{0,1\}^*$

$\Sigma^* = \{0,1\}^*$

Complexity Class P

Using formal language theory, we define

$$\mathsf{P} = \{L \subseteq \{0,1\}^* : \text{there exists an algorithm that decides } L \text{ in polynomial time}\}.$$

Theorem 34.2

$\mathsf{P} = \{L \subseteq \{0,1\}^* : \text{there exists an algorithm that accepts } L \text{ in polynomial time}\}.$

😲 This is surprising since *accepting* seems easier than *deciding*.

Proof of Theorem 34.2

It suffices to show: if L is accepted by a polynomial-time algorithm A, then L is also decided by some polynomial-time algorithm A'.

Proof of Theorem 34.2

It suffices to show: if L is accepted by a polynomial-time algorithm A, then L is also decided by some polynomial-time algorithm A'.

Since A accepts L in $O(n^k)$ time, there exists a constant c such that A accepts any $x \in L$ within at most cn^k steps. $\leq c \cdot n^k$.

After exactly cn^k steps, exactly one of two cases holds:

- A has already output 1, so $x \in L$.
- A is still running. Since any $x \in L$ would have been accepted by step cn^k, we conclude $x \notin L$. $\rightarrow$ output 0.

Proof of Theorem 34.2

It suffices to show: if L is accepted by a polynomial-time algorithm A, then L is also decided by some polynomial-time algorithm A'.

Algorithm A' simulates exactly cn^k steps of A:

- If A outputs 1 during the simulation, A' outputs 1.
- If A has not halted after cn^k steps, A' outputs 0.

A' runs in polynomial time (simulating cn^k polynomial-time steps adds at most a polynomial overhead), and it correctly accepts every $x \in L$ and rejects every $x \notin L$.

ITA 34.2 Polynomial-Time Verification

ITA 34.2 Polynomial-Time Verification

Verification Algorithms

📌 Example: HAM-CYCLE

Consider the language

encoding of G.

$$\texttt{HAM-CYCLE} = \{\langle G \rangle : G \text{ is hamiltonian}\}$$

😩 HAM-CYCLE is NP-complete, so no polynomial-time decision algorithm is known.

No known algorithm decides HAM-CYCLE in polynomial time.

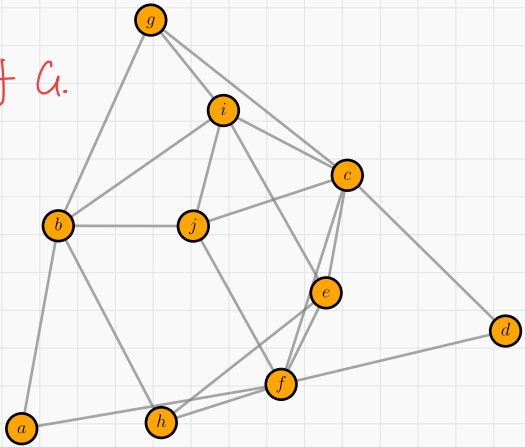


Figure 1: 🍰 Is $\langle G \rangle \in$ HAM-CYCLE?

📌 Example: HAM-CYCLE

Consider the language

$$\texttt{HAM-CYCLE} = \{\langle G \rangle : G \text{ is hamiltonian}\}$$

🙂 But given $\langle G \rangle$ and a permutation of its vertices

$$\langle v_1, v_2, \dots v_{|G.V|} \rangle$$

as a certificate, we can *verify* if it shows that

$$\langle G \rangle \in \texttt{HAM-CYCLE}$$

in polynomial time.

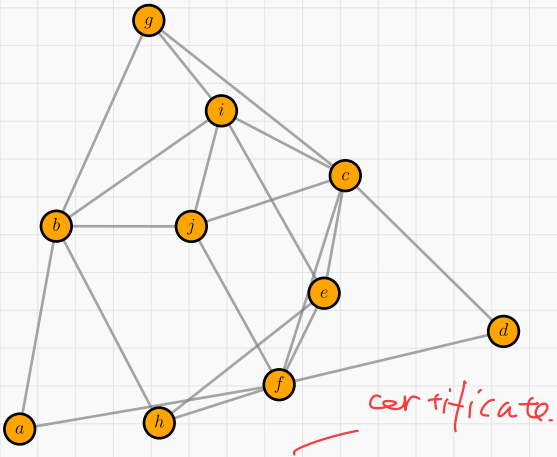


Figure 1: Is $\langle a, b, i, j, c, e, f, h, d, g \rangle$ a hamiltonian cycle?

Answer: No.

📌 Example: HAM-CYCLE

Consider the language

$$\texttt{HAM-CYCLE} = \{\langle G \rangle : G \text{ is hamiltonian}\}$$

🙂 But given $\langle G \rangle$ and a permutation of its vertices

$$\langle v_1, v_2, \dots v_{|G.V|} \rangle$$

as a certificate, we can *verify* if it shows that

$$\langle G \rangle \in \texttt{HAM-CYCLE}$$

in polynomial time.

The algorithm

$$A(\langle G \rangle, \langle v_1, \dots, v_n \rangle)$$

that returns

- 1 if $v_1, \dots, v_n$ is a hamiltonian cycle in G
- 0 otherwise

is a verification algorithm.

Verification Algorithms

A verification algorithm A takes two inputs:

- x — the problem instance, e.g., $\langle G \rangle$
- y — a certificate (a claimed "proof"), e.g., the sequence $\langle v_1, \dots, v_n \rangle$

A verifies x if there exists a y such that $A(x, y) = 1$.

The language verified by A is

$$L = \{x \in \{0,1\}^* : \exists y \in \{0,1\}^*,\ A(x, y) = 1\}.$$

💡 A verification algorithm need only *check* a given certificate, not search for one—this is why verification can be easier than decision.

ITA 34.2 Polynomial-Time Verification

The Complexity Class NP

Complexity Class NP

A language L belongs to NP if and only if there exist a polynomial-time algorithm A and a constant c such that

$$L = \{x \in \{0,1\}^* : \exists y \in \{0,1\}^*,\ A(x,y) = 1 \ \wedge\ |y| = O(|x|^c)\}.$$

The certificate can not be too big compared to $|x|$.

Complexity Class NP

A language L belongs to NP if and only if there exist a polynomial-time algorithm A and a constant c such that

$$L = \{x \in \{0,1\}^* : \exists y \in \{0,1\}^*,\ A(x,y) = 1 \ \wedge\ |y| = O(|x|^c)\}.$$

We say L can be verified in polynomial time by A.

📌 HAM-CYCLE $\in$ NP —

$$\begin{aligned}\textsc{HAM-CYCLE} = \{\langle G\rangle \in \{0,1\}^* :\ & \\ & \exists \langle v_1, \dots, v_{|G.V|}\rangle \in \{0,1\}^*, \\ & A(\langle G\rangle, \langle v_1, \dots, v_{|G.V|}\rangle) = 1\}\end{aligned}$$

A checks if $\langle v_1 \cdots v_{|n|}\rangle$ is a hamiltonian cycle

poly-time.

where A check if $v_1, \dots, v_{|G.V|}$ is a hamiltonian cycle in G.

So HAM-CYCLE $\subseteq$ NP

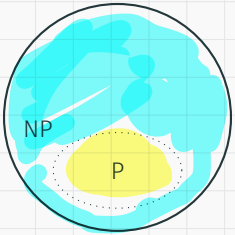
P ⊆ NP

Any polynomial-time decision algorithm can be converted into a polynomial-time verification algorithm by simply ignoring the certificate.

Formally, if A decides L in polynomial time, define $A'(x, y) = A(x)$. Then A' verifies L in polynomial time.

$$\begin{cases} A(x) = 1 & \text{if } x \in L \\ A(x) = 0 & \text{if } x \notin L \end{cases}$$

Therefore P ⊆ NP.



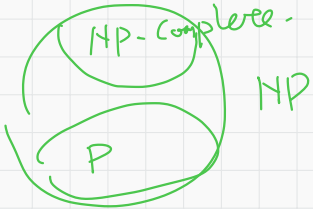
P ⊆ NP

Any polynomial-time decision algorithm can be converted into a polynomial-time verification algorithm by simply ignoring the certificate.

Formally, if A decides L in polynomial time, define $A'(x, y) = A(x)$. Then A' verifies L in polynomial time.

Therefore P ⊆ NP.

😖 Whether P = NP has been an open question since 1971.



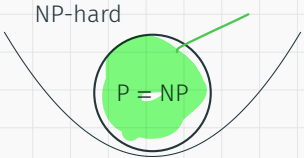


Figure 1: The case P = NP

Everything in NP can be decided in poly-time.

🤪 Beyond P = NP

Let co-NP = $\{L : \overline{L} \in \text{NP}\}$.

😢 We do not even know which of the four pictures below is true.

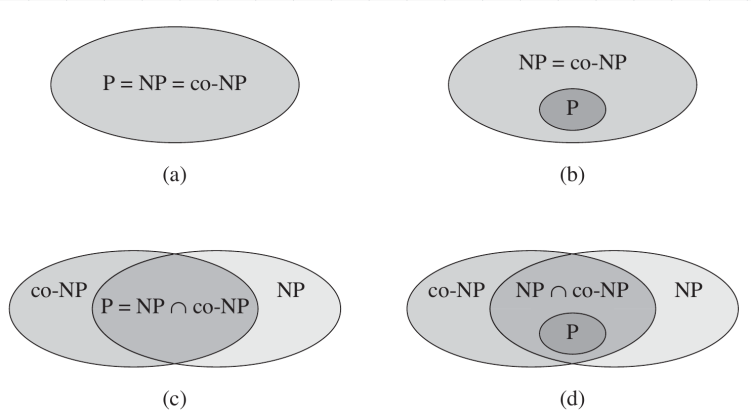


Figure 2: Four possible relationships among P, NP, and co-NP.

ITA 34.3 NP-Completeness and Reducibility

ITA 34.3 NP-Completeness and Reducibility

Reducibility

Reducibility

The problem

Is x a good 👨‍🎓?

can be answered by asking

Does x have GPA = 4.0?

in a hypothetical world where

good = high GPA

reduction

Figure 3: 🐢 — I am the fastest runner in class because I got a 💯 in calculus.

Reducibility

A problem Q can be reduced to a problem Q' if an instance of Q can be *easily* rephrased as an instance of Q'.

So instead of trying to answer Q, we can try to answer Q'.

🤔 What does *easily* mean here?

Figure 3: 🐢 — I am the fastest runner in class because I got a 💯 in calculus.

Polynomial-Time Reducibility

Let L_1 and L_2 be two languages.

We say that L_1 can be reduced in polynomial time to L_2, written

$$L_1 \leq_p L_2,$$

Polynomial-Time Reducibility

Let L_1 and L_2 be two languages.

We say that L_1 can be reduced in polynomial time to L_2, written

$$L_1 \leq_p L_2,$$

if there exists a function $f : \{0,1\}^* \to \{0,1\}^*$ such that

- f is computable in polynomial time, and
- $x \in L_1$ if and only if $f(x) \in L_2$.

The function f is called a reduction function.

An algorithm F that computes f in polynomial time is called a reduction algorithm.

Lemma 34.3

If $L_1 \leq_p L_2$ and $L_2 \in \mathsf{P}$, then $L_1 \in \mathsf{P}$.

Lemma 34.3

If $L_1 \leq_p L_2$ and $L_2 \in \mathsf{P}$, then $L_1 \in \mathsf{P}$.

Proof: If A_2 decides L_2 in polynomial time, then the algorithm A_1 shown below decides L_1 in polynomial time.

Reduction in poly time.

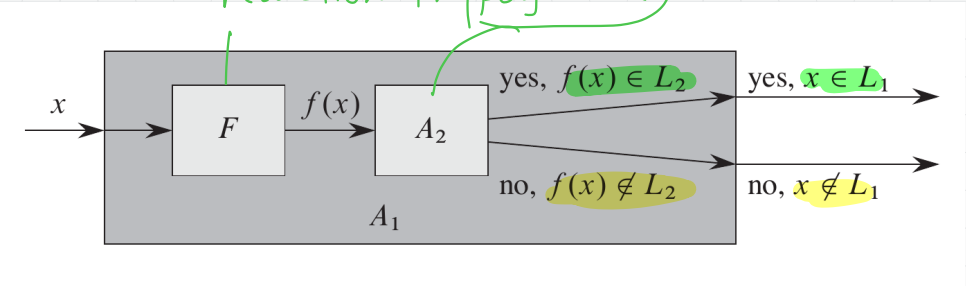


Figure 4: Algorithm A_1

$x \in L_1 \Longleftrightarrow f(x) \in L_2.$

Coming Up: Lecture 27

Using $\leq_p$, Lecture 27 will prove that a few well-known problems are NP-complete.

Each arrow is a polynomial-time reduction $L' \leq_p L$, transferring NP-hardness from L' to L.

💡 Only one problem needs to be shown NP-complete from scratch — every other follows by reduction.

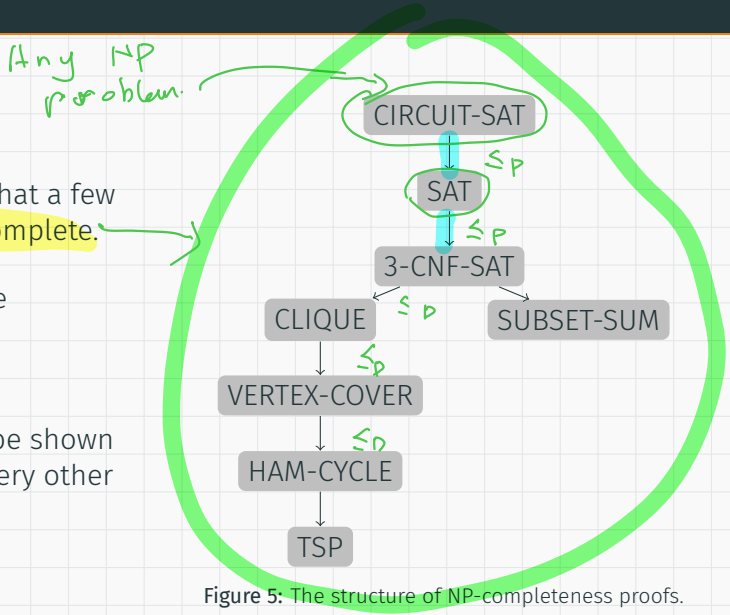


Figure 5: The structure of NP-completeness proofs.

What are your main takeaways today? Any questions?